

outputs a probability distribution over output variables $O_1, \dots, O_L$ (i.e. a probability distribution over the cross product of the domains specified by D_i). It thus maps an input substitution σ for the variables $I_1, \dots, I_m$ to a set of output substitutions θ_j with probability p_j . The neural rule serves as a template. For every input substitution σ , the template $(nt \rightarrow g_1, \dots, g_n)\sigma$ defines the set of instantiated stochastic definite clause grammar rules $p_j :: (nt \rightarrow g_1, \dots, g_n)\sigma\theta_j$.

Example 3 (Neural definite clause grammar rules). Consider the SDCG in Example 1. We can substitute the $n(X) \rightarrow [X]$ rules with the following neural rule

$$nn(mnist, [Mnist], [N], [digit]) :: n(N) \rightarrow [Mnist].$$

Here, the neural network called *mnist* takes as input the *Mnist* image and returns a probability for every possible number between 0 and 9, indicating how likely it is that every number is for the given MNIST image [20]. The predicate *digit* is defined as $digit(0), digit(1), \dots, digit(9)$. Given the neural network and the input substitution $\sigma = \{\mathbf{Mnist} = \text{img}\}$ (which could be obtained through unification with the terminal sequence), the neural network could generate the output substitutions $\theta_0 = \{N = 0\}; \dots; \theta_9 = \{N = 9\}$; with probabilities 0.87; ... ; 0.05. Thus, the neural rule with the input substitution $\sigma = \{\mathbf{Mnist} = \text{img}\}$ denotes the following set of grammar rules: $0.87 :: n(0) \rightarrow \text{img}; \dots; 0.05 :: n(9) \rightarrow \text{img}$

The neural rules are reminiscent of the neural predicates in DeepProbLog [23], which also encapsulate a neural network that outputs a distribution over a number of alternatives. It is worth analyzing how a neural rule behaves w.r.t the neural inputs (e.g. images). In fact, a neural rule defines a probability distribution over the values of the output variables *given* the neural inputs, whose distribution is not modeled in the program. This *conditional* setting is akin to *conditional* PCFGs [30, 39] and it is a common modeling strategy in discriminative parsing².

4 Inference in DeepStochLog

The goal of the inference is to compute the probability $P_G(\text{derives}(G, T))$ for a given goal G and (possibly unknown) sequence T . This is divided into two steps, a logical and probabilistic one, which we now explain.

Logical inference Given a DeepStochLog program, a goal G and a (potentially unknown) sequence T , logical inference uses resolution to answer $\text{derives}(G, T)$ ³. This corresponds to finding all the possible derivations for G that result in a terminal sequence T . The resolution process is then turned into a compact AND-OR circuit, which represents all possible derivations and will be the input for the probabilistic inference. The logical inference procedure is illustrated in Figure 1, where the SLD resolution tree for the given goal is on the left and its corresponding AND-OR circuit on the right. The translation to the AND-OR circuit is straightforward. It has exactly the same structure as the SLD-tree. For every resolution step with a rule $p_i : r_i$, an AND node is added. Furthermore, for a normal SDCG rule, the corresponding probability p_i is added, and for a neural grammar rule, there is a call to the neural network that returns the probability p_m . Whenever there are two (or more) branches in the SLD tree for a goal, an OR node is added. Notice that all the leaves are either probabilities given as parameters or the result of a neural call.

During SLD resolution, many identical intermediate goals may be proved multiple times which results in an explosion of inference time. To avoid proving the same goals, we use SLG resolution [4], which plays a similar role as the dynamic programming CYK algorithm for CFGs [16]. Tabling using SLG resolution is a standard logic programming technique that memoizes the answers of predicates by tabling the evaluations. This technique is incorporated in Prolog implementations such as XSB, SWI-Prolog and Prism [34]. The important difference with SLD resolution is that

²DeepStochLog could also be used to define generative grammars on subsymbolic inputs (e.g. images) if provided with neural models that can provide a joint probability distribution of both outputs and images. This is also discussed in [24] but will not be further analyzed in the current paper.

³This is sometimes called *phrase* or *sentence* in actual Prolog implementations and it requires an automatic syntactical translation of a DCG into Prolog. We show an example in Appendix C.

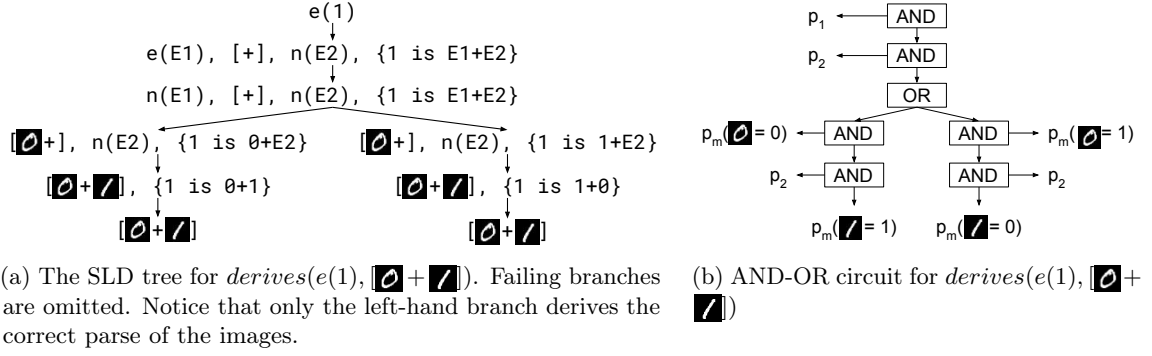


Figure 1: The different steps of inference on an example grammar.

the results are not a single derivation tree, but rather a forest, where certain parts are re-used for multiple derivations thanks to tabled evaluations. The effect of tabling is carried over to the creation of the AND-OR tree. Each time a derivation is reused from the table, its corresponding node is returned and linked to the new derivation. Thus, also the AND-OR circuit turns into a forest. When a goal admits a finite set of answers, we can resolve it only once and cache the corresponding AND-OR tree.

Probabilistic inference. With probabilistic inference, we refer to the task of calculating the probability $P(\text{derives}(G, T)) = \sum_{d(G\theta)=T} P(d(G\theta)) = \sum_{d(G\theta)=T} \prod_{r_i \in d(G\theta)} p_i^{m_i}$, i.e. the sum of the probabilities of all derivations for a given G that result in a given terminal sequence T and answer substitution θ . Thanks to SLG resolution and tabling, the shared sub-structure of many derivations is explicit in the AND-OR circuit obtained from the logical inference. This dissipates the need for a specialized algorithm, like the *inside* algorithm used in the probabilistic extension of CYK. Computing the probability $P(\text{derives}(G\theta, T))$ is just a bottom-up evaluation of the AND-OR circuit where AND-nodes are substituted by multiplications and OR-nodes by summations, i.e. compiling the logical circuit to an arithmetic circuit using the $(+, \times)$ semiring [17]. Analogously, the most probable derivation for the goal G is found with the $(\max, \times)$ semiring.

5 Learning in DeepStochLog

Learning in DeepStochLog is achieved by optimizing the parameters of the neural networks and the parameters of the logic programs itself. Let us consider a dataset of triples $\mathcal{D} = \{(G_i\theta_i, T_i, t_i)\}$, where G_i is a goal, θ_i a substitution for G_i , T_i a sequence of terminals and t_i a target probability. Let us also consider a DeepStochLog program parameterized by the vector p of rule probabilities. Learning in DeepStochLog is defined as the following optimization problem, with $\mathcal{L}$ being any differentiable loss function:

$$\min_p \sum_{(G_i\theta_i, T_i, t_i) \in \mathcal{D}} \mathcal{L}\left(P_G(\text{derives}(G_i\theta_i, T_i); p), t_i\right) \quad (1)$$

Representing the dynamic programming computation for the *inside* probability in terms of an arithmetic circuit has an important advantage. In fact, the corresponding computational graph is differentiable and the derivatives of the loss function $\mathcal{L}$ w.r.t. the probabilities p can be carried out automatically using out-of-the-box differentiation frameworks. Moreover, when the probabilities p are computed by a neural network as for a neural grammar rule, the gradients can be seamlessly backpropagated to the network to train its internal parameters. We solve the learning problem using standard gradient descent techniques from deep learning, e.g. the Adam optimizer [18].

One interesting case is when the loss function $\mathcal{L}$ is the negative log-likelihood, as it brings DeepStochLog into the standard learning scenario for probabilistic grammars. Here, the optimization problem is usually carried out in the expectation-maximization (EM) framework. Given an *inside* algorithm, a correspondent *outside* algorithm is designed to extract the expected counts of the

various grammar rules from data (E-step) and then the counts are used to update the probabilities (M-step). Most of the developed inside-outside algorithms are tailored to a specific formalism. However, the gradient descent approach of DeepStochLog on the negative log-likelihood is actually equivalent to the EM approach but it does not require the explicit definition of the corresponding outside algorithm. In fact, the gradients obtained by the backward pass through the AND-OR circuit have been shown to actually compute the outside probabilities (E-step), while the gradient descent step is used to update the parameters (M-step) [33, 2, 13].

6 Evaluation

6.1 Research Questions

The goal of our experiments is to answer the following questions:

- Q1** Does DeepStochLog reach state-of-the-art predictive performance on neural-symbolic tasks?
- Q2** How does the inference time of DeepStochLog compare to other neural-symbolic frameworks and what is the role of tabling?
- Q3** Can DeepStochLog handle larger-scale tasks?
- Q4** Can DeepStochLog go beyond grammars and encode more general programs?

6.2 Tasks

We specify the tasks used in this paper. Complete details are specified in Appendix B.

T1: MNIST Addition. In the MNIST Addition task [23], the model is given two sequences of length N of MNIST images containing handwritten images [20], each representing an N -digit number. The task is to predict the sum of these numbers (  +   = 56). The training data only contains the two image sequences and the sum of the corresponding numbers, thus not providing the digit labels of the individual images. The datasets for each digit length use all 60K images of MNIST images exactly once.

T2: Handwritten Formulas. In the Handwritten Formulas (HWF) task, the goal is to solve mathematical expressions, where both digits and operators (addition, subtraction, multiplication and division) are images of handwritten characters, like     . Like T1, the data of T2 only contains the outcome of the expression and the sequence of images. For this task, we use the Handwritten Formula (HWF) dataset, introduced in [21]. The dataset contains 10000 expressions of lengths 1, 3, 5 and 7. Unlike the original paper, we do not consider a curriculum learning setting here, and split the dataset into 4 separate parts by length.

T3: Well-formed Parentheses. We introduce the Well-formed Parentheses task, where the model is asked to recognize image sequences that represent well-formed parentheses. Well-formed parentheses is a classic context-free grammar language where $\Sigma = \{ (,) \}$, and $R = \{ s \rightarrow () | (s) | ss \}$, i.e. all open brackets are closed in the right order. As images, we use the zeros from MNIST as “(” and ones as “)”, and generate 1000 well-formed parenthesis sequences without labels as training data. The goal is to predict the most probable parse of the bracket sequence.

T4: Context-Sensitive Grammar. Since DCGs support context-sensitive grammars, we created a dataset of 2000 image sequences representing the canonical context-sensitive grammar $a^n b^n c^n$. Since each sequence of length $3n$ only has one valid parse, we increased the difficulty by allowing permutations such as $b^n a^n c^n$. We also generated 2000 negative examples, i.e. random sequences of the form $a^k b^l c^m$, and permutations like $c^k a^l b^m$ such that k, l, m are all larger than 1, sum to a multiple of 3 and are not all the same number. The goal of the task is to recognize whether the input sequence belongs to the first grammar or the second.

T5: Semi-supervised classification in citation networks. Given a set of scientific papers represented as bag-of-words and their citation network, the goal is to assign the correct class to a large test set of documents by having access only to the true labels of a small training set. The intuition is that one can infer the class of a paper not only by the features of the document but also by the class of its neighbors. This task is interesting from a neural symbolic perspective because one